

Client-side Development in ASP.NET Core4

Client-side development refers to the code that runs inside the user's web browser. It is mainly used to make web pages interactive, responsive, and user-friendly.

Common Client-side Web Technologies

Technology	Definition	Example
HTML	Defines the structure and content of a web page.	Headings, forms, tables
CSS	Controls the appearance and layout of a web page.	Colors, fonts, spacing
JavaScript	Adds logic and interactivity to web pages.	Button click, calculations
jQuery	A JavaScript library that makes DOM manipulation, events, and AJAX easier.	<code>\$("#msg").text("Hello");</code>
Bootstrap	A CSS framework used to create responsive and attractive interfaces.	Responsive buttons, menus

```
<h2 id="msg">Welcome</h2>
<button onclick="changeText()">Click Me</button>

<script>
function changeText() {
    document.getElementById("msg").innerHTML = "Hello Student!";
}
</script>
```

Output: When the button is clicked, **Welcome** changes to **Hello Student!**

jQuery

jQuery is a JavaScript library that provides simple methods for manipulating HTML elements, handling events, animations, and making AJAX requests.

```
<p id="message">Hello</p>
<button id="btn">Click Me</button>

<script src="https://code.jquery.com/jquery-3.7.1.min.js"></script>

<script>
$("#btn").click(function () {
    $("#message").text("Welcome to jQuery!");
});
</script>
```

Output:

Before clicking:

Hello [Click Me]

After clicking:

Welcome to jQuery!

Common jQuery functions

- `$("#id")` → Selects an HTML element.
- `.text()` → Changes text.
- `.hide()` → Hides an element.
- `.show()` → Shows an element.
- `.click()` → Handles button click events.
- `.val()` → Gets or sets form values.

Example: Show/Hide Message using jQuery

1. Create Index.cshtml :

HTML

```
@{
    ViewData["Title"] = "jQuery Example";
}

<h2>jQuery Example</h2>

<button id="showBtn">Show Message</button>
<button id="hideBtn">Hide Message</button>

<p id="message">Welcome to ASP.NET Core with jQuery!</p>

<script src="https://code.jquery.com/jquery-3.7.1.min.js"></script>
```

```
<script>
    $(document).ready(function () {

        $("#showBtn").click(function () {
            $("#message").show();
        });

        $("#hideBtn").click(function () {
            $("#message").hide();
        });

    });
</script>
```

Design a client side validation application using jquery for taking input data from student in Library Management System, assume your own assumptions.

Assumption: The library collects Student Name, Email, Book Name, and Phone Number.

```
<html>
<head>
  <title>Library Management System</title>

  <!-- jQuery -->
  <script src="https://code.jquery.com/jquery-3.7.1.min.js"></script>
</head>

<body>

<h2>Library Management System</h2>

<form id="libraryForm">

  Student Name:
  <input type="text" id="name"><br><br>

  Email:
  <input type="text" id="email"><br><br>
```

```
    Book Name:
    <input type="text" id="book"><br><br>

    Phone:
    <input type="text" id="phone"><br><br>

    <button type="submit">Submit</button>

  </form>

  <p id="message"></p>
```

```

<script>
$(document).ready(function () {

    $("#libraryForm").submit(function (event) {

        event.preventDefault();

        let name = $("#name").val();
        let email = $("#email").val();
        let book = $("#book").val();
        let phone = $("#phone").val();

        if (name == "") {
            $("#message").text("Student Name is required.");
        }
        else if (email == "" || !email.includes("@")) {
            $("#message").text("Enter a valid Email.");
        }

```

```

        else if (book == "") {
            $("#message").text("Book Name is required.");
        }
        else if (phone == "" || phone.length != 10) {
            $("#message").text("Enter a valid 10-digit Phone Number.");
        }
        else {
            $("#message").text("Student data submitted successfully!");
        }
    });
});
</script>

</body>
</html>

```

Validation performed

- Student Name → Cannot be empty.
- Email → Must contain @.
- Book Name → Cannot be empty.
- Phone → Must contain 10 digits.

Single Page Application (SPA)

A Single Page Application (SPA) is a web application that loads one main web page and dynamically updates its content without completely reloading the page whenever the user interacts with it.

The page does not refresh again and again; only the required part of the page is updated.

Example

Suppose we have an online library system:

Home | Books | Students | About

|
| List of Books |
|

When the user clicks Books, only the book list is changed. The entire webpage does not reload.

Common SPA Frameworks

Common SPA Frameworks

1. **Angular** – A framework developed by Google for building dynamic web applications.
2. **React** – A JavaScript library developed by Meta for building interactive user interfaces.

Advantages of SPA

- Faster navigation
- No complete page reload
- Better user experience
- Suitable for interactive web applications
- Uses APIs to communicate with the server

Angular is a TypeScript-based framework for developing dynamic web applications, while React is a JavaScript library for building interactive user interfaces and SPAs.

SPA = One web page + Dynamic content update without full page reload.

Angular

Angular is a TypeScript-based web framework developed by Google for building dynamic and single-page web applications.

Advantages of Angular

- *Complete framework – Provides tools for routing, forms, HTTP, etc.*
- *Good for large applications – Suitable for complex projects.*
- *TypeScript support – Helps reduce programming errors.*
- *Two-way data binding – Makes synchronization between UI and data easier.*
- *Component-based – Applications can be divided into reusable components.*

Disadvantages of Angular

- *Steep learning curve – Many concepts to learn.*
- *More complex than simpler JavaScript libraries.*
- *Large application size compared with some alternatives.*
- *Can be slower for very small applications.*
- *Requires TypeScript knowledge for effective development.*

React

React is a JavaScript library developed by Meta for building interactive and reusable user interfaces.

Advantages of React

- *Easy to learn – Especially for developers familiar with JavaScript.*
- *Reusable components – Components can be reused in different parts of an application.*
- *Fast performance – Uses a Virtual DOM to efficiently update the UI.*
- *Flexible – Can be combined with other libraries and tools.*
- *Large community – Many learning resources and third-party packages are available.*

Disadvantages of React

- *Only a UI library – Additional libraries may be needed for routing and other features.*
- *Many choices – Beginners may be confused about which libraries/tools to use.*
- *Frequent updates can require learning new approaches.*
- *Requires JavaScript/JSX knowledge.*
- *Project structure is less standardized than Angular.*

Write a Angular program to display a list of students using Angular.

app.component.ts

 TypeScript

```
export class AppComponent {  
  students = ["Ram", "Sita", "Hari", "Gita"];  
}
```

Here:

- `students` is an array containing student names.
- `*ngFor` repeats the `<li>` element for every student.
- `{{ student }}` displays the student name.

app.component.html

 HTML

```
<h2>Student List</h2>  
  
<ul>  
  <li *ngFor="let student of students">  
    {{ student }}  
  </li>  
</ul>
```

Output:

Student List

- Ram
- Sita
- Hari
- Gita

App.js

JavaScript

```
import { useState } from "react";

function App() {
  const [message, setMessage] = useState("Hello Student");

  return (
    <div>
      <h2>{message}</h2>

      <button onClick={() => setMessage("Welcome to React!")}>
        Click Me
      </button>
    </div>
  );
}

export default App;
```

Write a React program that changes a message when a button is clicked.

Here:

- `useState()` stores the message.
- `message` contains the current text.
- `setMessage()` changes the text.
- `onClick` executes the function when the button is clicked.

Simple difference: Angular uses directives such as `*ngFor`, while React commonly uses JavaScript/JSX expressions and state such as `useState()`.

Validation in Angular Framework

Assumption

We create a simple **Library Management System** form that accepts:

- Student Name
 - Email
 - Book Name
 - Phone Number
- `ng` → A prefix commonly used by Angular for its directives and commands, such as `ngModel`, `ngIf`, and `ngFor`.
 - `ngModel` → An Angular directive used for **two-way data binding** between a form input and a component variable.

Step 1: Create Angular Project

`</>` **Bash**

```
ng new library-app
cd library-app
ng serve
```

Then replace the contents of the main component with the following.

Step 2: app.component.ts

</> TypeScript

```
import { Component } from '@angular/core';
import { FormsModule } from '@angular/forms';

@Component({
  selector: 'app-root',
  standalone: true,
  imports: [FormsModule],
  templateUrl: './app.component.html'
})
export class AppComponent {

  submitForm(form: any) {

    if (form.valid) {
      alert("Student data submitted successfully!");
    } else {
      alert("Please fill all required fields.");
    }
  }

}
```

Step 3: app.component.html

HTML

```
<label>Book Name:</label>
<input type="text"
       name="book"
       [(ngModel)]="bookName"
       required>

<br><br>

<label>Phone:</label>
<input type="text"
       name="phone"
       [(ngModel)]="phone"
       required
       minlength="10"
       maxlength="10">

<br><br>

<button type="submit">Submit</button>

</form>
```

React:

React is a JavaScript library developed by Meta for building interactive user interfaces and Single Page Applications.

Angular	React
Full web framework	JavaScript UI library
Developed by Google	Developed by Meta
Uses TypeScript	Mainly uses JavaScript/JSX
Used for SPAs	Used for SPAs and interactive UIs

Validation Example using React

```

import { useState } from "react";

function StudentForm() {

  const [name, setName] = useState("");
  const [email, setEmail] = useState("");

  function submitForm(e) {
    e.preventDefault();

    if (name === "" || email === "") {
      alert("Please enter all fields.");
    } else {
      alert("Form submitted successfully!");
    }
  }

  return (
    <form onSubmit={submitForm}>

      <h2>Student Form</h2>

```

```

      Student Name:
      <input
        type="text"
        onChange={(e) => setName(e.target.value)}
      />

      <br /><br />

      Email:
      <input
        type="email"
        onChange={(e) => setEmail(e.target.value)}
      />

      <br /><br />

      <button type="submit">Submit</button>

    </form>
  );
}

export default StudentForm;

```

What is TempData? How is it different from Session state and ViewData? When would you use TempData over other state management techniques

6

What does single page application means? Discuss about server side strategies.

2. How do you manage state on stateless HTTP? How do you validate form using JQuery? Describe with your own assumptions. [4 + 6]

6

What does single page application means? Discuss about server side strategies.

Define state management. Explain the parts of state management on the client side.

7

Why do we need to validate the form? How can you use react for this?

Give an example to manage the session state.

Compare Angular and React frameworks for building Single Page Applications (SPA) with ASP.NET Core backend. What are the advantages and disadvantages of each?

7

Write a program to validate the form that takes input user name, password and phone number using JQuery. All fields must be non-empty and phone number must contain exactly 10 digits.



Explain the use of Cache as a server-side state management strategy. What are the benefits of caching in web applications?

Define state management. Explain the parts of state management on the client side.

Why do we need to validate the form? How can you use react for this?